# AI Assisted Coding Lab ASS-13.4

Name: P.Tharun Kumar

Batch:14

2303A510F1

## Task 1: Refactoring Data Transformation

## Logic

Scenario

You are maintaining a data preprocessing script were numerical transformations are written using verbose loops.

Task Description

• Review the legacy code that computes transformed values using an explicit loop.

• Use an AI tool to suggest a more Pythonic refactoring approach.

• Refactor the code using list comprehensions or helper functions while preserving the output.

Legacy Code

```
values = [2, 4, 6, 8, 10]
doubled = []
for v in values:
doubled. Append (v * 2)
print(doubled)
```

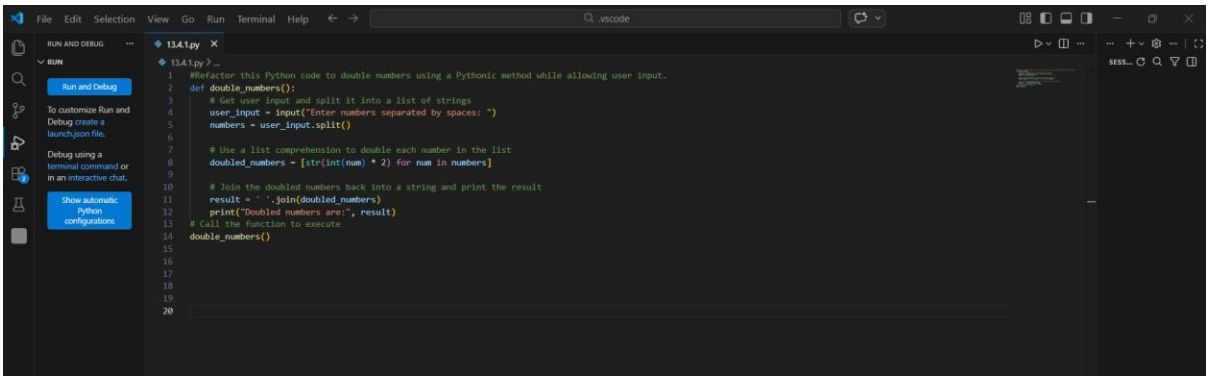
Expected Output

[4, 8, 12, 16, 20]

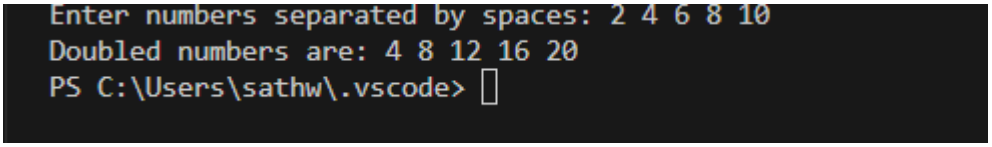
PROMPT:

#Refactor this Python code to double numbers using a Pythonic method while allowing user input.

CODE:




OUTPUT:




## Task 2: Improving Text Processing Code

## Readability

Scenario

You are working on a log message generator that builds messages word by word.

Task Description

• Analyse the legacy code that constructs a sentence using repeated string concatenation.

• Ask AI to suggest a more efficient and readable approach.

• Refactor the code accordingly while keeping the final output unchanged.

Legacy Code

```
words = ["Refactoring", "with", "AI", "improves",
"quality"]
message = ""
```

for w in words:

message += w + " "

print (message. Strip ())

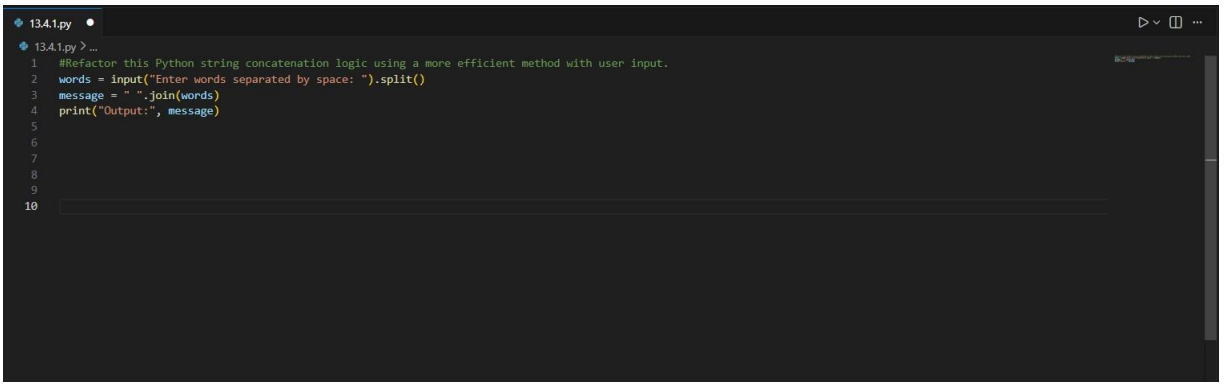
Expected Output

Refactoring with AI improves quality

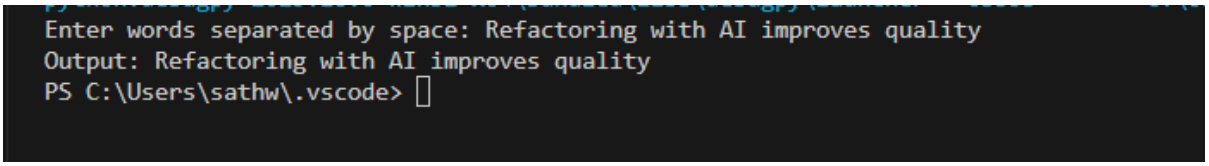
PROMPT:

#Refactor this Python string concatenation logic using a more efficient method with user input.

CODE:




OUTPUT:




Task 3: Safer Access to Configuration Data

Scenario

You are maintaining a configuration loader where dictionary keys may

or may not exist depending on deployment settings.

Task Description

• Review the legacy code that manually checks for dictionary keys.

• Use AI suggestions to refactor the code using safer dictionary

access methods.

• Ensure the behaviour remains the same for missing keys.

Legacy Code

```
config = {"host": "localhost", "port": 8080}
if "timeout" in config:
print(config["timeout"])
else:
print ("Default timeout used")
```

Expected Output

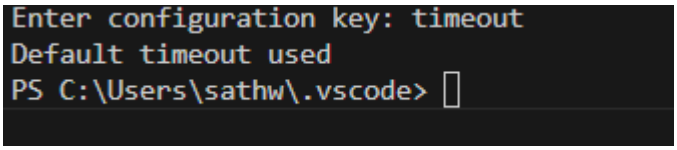
Default timeout used

PROMPT:

#Refactor dictionary key checking using a safer method and allow the user to enter the key.

CODE:


OUTPUT:




---

Task 4: Refactoring Conditional Logic for

Scalability

Scenario

You are enhancing a utility module that performs different operations

based on user input.

Task Description

• Examine the multiple if–Elif conditions used to determine

operations.

• Ask AI to suggest a cleaner, scalable alternative.

• Refactor the logic using mapping techniques while preserving

functionality.

Legacy Code

```
action = "divide"
x, y = 10, 2
if action == "add":
result = x + y
Elif action == "subtract":
result = x - y
Elif action == "multiply":
result = x * y
Elif action == "divide":
result = x / y
else:
result = None
print(result)
```

Expected Output

5.0

PROMPT:

#Refactor this Python operation logic using dictionary mapping and allow the user to enter operation and numbers.

CODE:

```python
#Refactor this Python operation logic using dictionary mapping and allow the user to enter operation and numbers.
def add(x, y):
    return x + y

def subtract(x, y):
    return x - y
def multiply(x, y):
    return x * y
def divide(x, y):
    if y == 0:
        return "Error: Division by zero is not allowed."
    return x / y
operations = {
    'add': add,
    'subtract': subtract,
    'multiply': multiply,
    'divide': divide
}
def calculator():
    operation = input("Enter the operation (add, subtract, multiply, divide): ")
    if operation not in operations:
        print("Invalid operation. Please try again.")
        return
    try:
        num1 = float(input("Enter the first number: "))
        num2 = float(input("Enter the second number: "))
    except ValueError:
        print("Invalid input. Please enter numeric values.")
        return
    result = operations[operation](num1, num2)
    print(f"The result of {operation}ing {num1} and {num2} is: {result}")
calculator()
```

OUTPUT:

```
Enter the operation (add, subtract, multiply, divide): subtract
Enter the first number: 20
Enter the second number: 10
The result of subtracting 20.0 and 10.0 is: 10.0
PS C:\Users\sathw\.vscode> ^C
PS C:\Users\sathw\.vscode>
PS C:\Users\sathw\.vscode>  c:; cd 'c:\Users\sathw\.vscode'; & 'c:\Users\sathw\AppData\Local\Microsoft\WindowsApps\python3.13.exe' 'c:\Users\sathw\.vscode\extensions\ms-
python.debugpy-2025.18.0-win32-x64\bundled\libs\debugpy\launcher' '49393' '--' 'C:\Users\sathw\.vscode\13.4.1.py'
Enter the operation (add, subtract, multiply, divide): multiply
The result of subtracting 20.0 and 10.0 is: 10.0
PS C:\Users\sathw\.vscode> ^C
PS C:\Users\sathw\.vscode>
PS C:\Users\sathw\.vscode>  c:; cd 'c:\Users\sathw\.vscode'; & 'c:\Users\sathw\AppData\Local\Microsoft\WindowsApps\python3.13.exe' 'c:\Users\sathw\.vscode\extensions\ms-
python.debugpy-2025.18.0-win32-x64\bundled\libs\debugpy\launcher' '49393' '--' 'C:\Users\sathw\.vscode\13.4.1.py'
Enter the operation (add, subtract, multiply, divide): multiply
PS C:\Users\sathw\.vscode>
PS C:\Users\sathw\.vscode>  c:; cd 'c:\Users\sathw\.vscode'; & 'c:\Users\sathw\AppData\Local\Microsoft\WindowsApps\python3.13.exe' 'c:\Users\sathw\.vscode\extensions\ms-
python.debugpy-2025.18.0-win32-x64\bundled\libs\debugpy\launcher' '49393' '--' 'C:\Users\sathw\.vscode\13.4.1.py'
Enter the operation (add, subtract, multiply, divide): multiply
python.debugpy-2025.18.0-win32-x64\bundled\libs\debugpy\launcher' '49393' '--' 'C:\Users\sathw\.vscode\13.4.1.py'
Enter the operation (add, subtract, multiply, divide): multiply
Enter the operation (add, subtract, multiply, divide): multiply
Enter the first number: 3
Enter the second number: 4
Enter the first number: 3
Enter the second number: 4
Enter the second number: 4
The result of multiplying 3.0 and 4.0 is: 12.0
PS C:\Users\sathw\.vscode> ^C
PS C:\Users\sathw\.vscode>
PS C:\Users\sathw\.vscode>  c:; cd 'c:\Users\sathw\.vscode'; & 'c:\Users\sathw\AppData\Local\Microsoft\WindowsApps\python3.13.exe' 'c:\Users\sathw\.vscode\extensions\ms-
PS C:\Users\sathw\.vscode>  c:; cd 'c:\Users\sathw\.vscode'; & 'c:\Users\sathw\AppData\Local\Microsoft\WindowsApps\python3.13.exe' 'c:\Users\sathw\.vscode\extensions\ms-
python.debugpy-2025.18.0-win32-x64\bundled\libs\debugpy\launcher' '49675' '--' 'C:\Users\sathw\.vscode\13.4.1.py'
Enter the operation (add, subtract, multiply, divide): divide
Enter the first number: 30
Enter the second number: 5
The result of divideing 30.0 and 5.0 is: 6.0
PS C:\Users\sathw\.vscode>
```

## Task 5: Simplifying Search Logic in Collections

### Scenario

You are reviewing legacy inventory-check code that uses manual loops to find elements.

Task Description

- Identify the explicit loop used for searching an item.

• Use AI assistance to refactor the logic into a more concise and

readable form.

• Maintain the same output behaviour.

Legacy Code

inventory = ["pen", "notebook", "eraser", "marker"]

found = False

for item in inventory:

if item == "eraser":

found = True

break

print ("Item Available" if found else "Item Not Available")

Expected Output

Item Available

PROMPT:

# Refactor this Python list search logic using a more concise method and allow the user to enter the item.

CODE:


OUTPUT: